VISHWAKARMA UNIVERSITY, PUNE

Faculty of Science & Technology

Department of Computer Engineering

A.Y. 2026–2027 | Term I

MAJOR PROJECT — BTECCE23706

REVIEW 2 REPORT

Parameter 2 — System Design & Development (20 Marks)

AI-POWERED PERSONAL LEARNING ASSISTANT

A document-grounded learning platform combining RAG, AI-generated practice, learning analytics and personalized study planning

Submitted to

Project Guide: Prof. Harish Motekar

Submitted by

Sr. No.	Roll No.	SRN	PRN	Student Name
1	45	31242273	3124045020163	Vishwajeet Yadav
2	46	31242274	3124045020164	Aayush Aherkar
3	47	31242293	3124045020169	Kshitij Rathod
4	48	31242303	3124045020172	Brijesh Ingavale
5	55	31242379	3124045020184	Shiv Bhosale

Date of Submission: October 2026

Abstract

The AI-Powered Personal Learning Assistant is proposed as a web-based platform that helps students work with their own academic notes and documents. As established in Review 1, the system is intended to support document upload, context-aware question answering through Retrieval-Augmented Generation (RAG), automatic summaries, multiple-choice questions, flashcards, viva preparation, quiz evaluation, weak-topic identification, previous question-paper analysis and personalized study planning. Review 2 develops the proposed system design into a more concrete, modular implementation plan. It describes the interaction between the frontend, Java Spring Boot backend, document-processing components, vector retrieval layer, LLM service and data stores; clarifies the responsibilities of the major modules; and identifies a manageable order for implementation.

The status recorded in this draft is deliberately conservative. The available Review 1 document describes the project as proposed, and the team has confirmed that implementation has not yet started. Accordingly, this report does not claim that a working application, live RAG pipeline, completed database, executed tests or observed production defects already exist. The milestone table distinguishes completed proposal work from design tasks and implementation tasks that remain outstanding. A separate section lists common technical issues that should be checked during development; these are anticipated risks, not defects observed in live execution.

1. Scope of This Review

Review 2 focuses on system design and the readiness of the project for implementation.

Review criterion	Section	Status represented in this report
System architecture	§3 — System Design	Layered design and end-to-end data flow elaborated; detailed design still requires confirmation.
Technology selection	§4 — Technology Selection	Review 1 stack retained; vector-store and LLM provider decisions remain open.
Implementation progress	§5 — Implementation Progress	Proposal/design work documented; application implementation has not started.
Innovation	§6 — Innovation	Integration of source-grounded study support, practice and learner progress into one workflow.
Technical complexity	§7 — Technical Complexity	Main integration challenges and practical ways to contain scope identified.
Milestone completion	§8 — Milestone Completion	Completed, partially completed and not-started work packages separated; revised schedule proposed.

Sections 9 to 12 describe the proposed testing approach, updated risk register, the appropriate Review 2 presentation walkthrough, and the path from the current design stage to the next review.

2. Progress Since Review 1

Review 1 established the problem statement, objectives, proposed architecture, module list, technology stack and broad development sequence. It also stated that implementation would take place in the subsequent development phases. The available project information does not document completed software modules between the two reviews.

Area	Position in Review 1	Position recorded for Review 2
Problem and objectives	Problem statement and ten project objectives documented.	Scope retained. The main user journey is expressed as Upload → Understand → Practise → Analyse → Improve.
Literature review	Research themes listed; detailed paper entries were still marked “To be added”.	Relevant RAG, educational AI and automatic question-generation sources have been identified for a preliminary literature matrix. A full critical survey remains outstanding.
Architecture	High-level frontend, Spring Boot backend, databases, RAG and LLM components proposed.	Layer responsibilities, module boundaries, request flow and data responsibilities elaborated in §3.
Database	Initial entities listed: USER, DOCUMENT, QUIZ, QUIZ_RESULT, CHAT_HISTORY, FLASHCARD, PERFORMANCE and STUDY_PLAN.	Entity purpose and relationships clarified conceptually. ER diagram, table definitions and migrations remain to be prepared.

Area	Position in Review 1	Position recorded for Review 2
Technology stack	HTML/CSS/JavaScript, Java/Spring Boot, MySQL, PDFBox, POI, LLM API, embeddings and vector storage proposed.	Core stack retained. Exact embedding/LLM service and vector-store option remain to be selected before coding.
Implementation	Proposed; implementation to follow in later phases.	Not started, according to the team's current status. No live application or working module is claimed.
Testing	Functional, integration, usability and performance testing proposed.	Test cases and acceptance checks are outlined in §9. Execution evidence is not yet available.

3. System Design

3.1 Proposed layered architecture

The proposed platform is organized into five logical layers. The purpose of this separation is to keep user-interface code, application workflows, AI-based learning functions, document retrieval and persistent storage understandable and independently testable. The architecture is intentionally a conventional web application rather than a distributed microservice system, which keeps the implementation within a practical scope for the team.

Layer	Responsibility	Proposed realization
L1 — Presentation	Accept student actions and display documents, answers, practice material and progress.	HTML, CSS and JavaScript pages for login, document library, study chat, quiz and dashboard.
L2 — Application backend	Handle authentication, validation, REST requests, ownership checks and coordination between modules.	Java with Spring Boot; REST APIs; Spring Security and JWT as proposed in Review 1.
L3 — Learning services	Coordinate document chat, content generation, quiz evaluation, performance summaries and study-plan creation.	Service classes within the Spring Boot application; one configured LLM API for the initial version.
L4 — Document and RAG processing	Extract text, prepare chunks, create embeddings, retrieve relevant passages and construct context for the LLM.	Apache PDFBox for PDF; Apache POI for Word/PowerPoint formats; embedding service and vector search.
L5 — Data storage	Persist application records and document-retrieval information.	MySQL for users, files, chats, quizzes and study plans; PostgreSQL with pgvector or another compatible vector store remains under evaluation for embeddings.

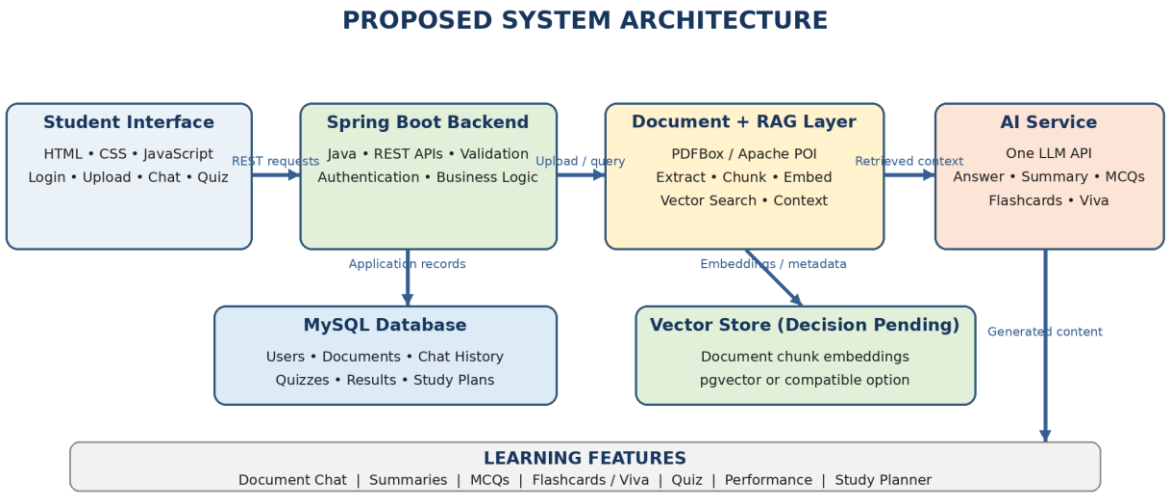


Figure 1. Proposed high-level architecture. The vector-storage option and external AI provider are decisions to be finalized before implementation.

3.2 Module decomposition

Module	Main responsibility	Expected interaction
User Management	Registration, login, authentication, profile and access control.	The backend identifies the logged-in student and limits access to that student's records.

Module	Main responsibility	Expected interaction
Document Management	Upload, store, list and manage PDF, PPT and Word study material.	A successful upload creates a document record and starts the processing workflow.
Document Processing	Extract text, clean it and split it into meaningful chunks.	PDFBox handles PDFs; Apache POI is proposed for Word and PowerPoint documents.
RAG and AI Study Chat	Retrieve passages relevant to the student's question and request a grounded response from the LLM.	The response should display available source-document information alongside the explanation.
Learning Content Generation	Generate summaries, MCQs, flashcards and viva questions from selected material.	Generated items should remain associated with the selected document or topic.
Quiz and Assessment	Present questions, record answers and calculate scores.	Quiz attempts and results are stored for later topic-wise analysis.
Performance Analytics	Summarize scores and identify topics where performance is comparatively weak.	A simple threshold-based approach can be used initially; a trained prediction model is not required.
Previous Question Paper Analysis	Identify repeated terms or topics and prepare a priority list from uploaded papers.	Initial version can use text extraction and frequency-based comparison rather than a complex predictive model.
Study Planner	Prepare a revision sequence using exam date, available hours, weak topics and learning progress.	A rule-based schedule can prioritize weaker topics and distribute revision activities.

3.3 Document processing and RAG flow

The RAG pipeline will use the student's uploaded material as an external knowledge source. The first implementation should prioritize text-based documents that can be processed reliably. The planned flow is:

1. Upload: the student selects a supported PDF, PowerPoint or Word document. The backend validates its file type and size and stores its metadata.
2. Extract: the document-processing layer extracts readable text using PDFBox or Apache POI, depending on the file format.
3. Clean and chunk: the extracted text is normalized and divided into smaller sections. Metadata such as document ID, page number or section label is retained where available.
4. Embed and index: each chunk is converted into a numerical embedding using the selected embedding service and stored with its metadata in the chosen vector store.
5. Retrieve: when the student asks a question, the system converts the query into an embedding and retrieves the most relevant chunks.
6. Generate: the backend sends the question and retrieved passages to the LLM with an instruction to answer from the supplied context and acknowledge when the context is insufficient.
7. Display and store: the interface displays the answer with available source-document references; the conversation and related metadata are stored in MySQL.

For example, a student uploads a lecture PDF on database normalization and asks, "What is the difference between 2NF and 3NF?" The retrieval layer should select the relevant sections from that PDF and pass them to the LLM. The response should explain the difference using the retrieved material and identify the source document or page where possible. If the uploaded material does not contain the answer, the assistant should state that the answer is not available in the selected notes rather than inventing a document-based citation.

3.4 Quiz, performance and study-plan flow

A student selects a document or topic and requests practice questions. The AI service generates a small set of questions, options and answer keys. The student attempts the quiz, and the backend calculates the score and stores the result with the related topic. Performance analysis can initially use a straightforward rule: topics with lower accuracy across recent attempts are marked for revision. The study planner then combines those topics with the examination date and the student's available study hours to generate a realistic sequence of reading, flashcard review, MCQ practice and revision.

The initial version does not require machine-learning-based prediction of student outcomes. A transparent rule-based approach is easier to build, explain during the review and validate using sample quiz records. More advanced personalization can remain future work.

3.5 Proposed database structure

Entity	Information to be stored	Relationship / purpose
USER	Student ID, name, email, password hash and account details.	One user can own multiple documents, chats, quiz attempts and study plans.
DOCUMENT	Owner ID, file name, file type, upload time and processing status.	Belongs to one user; links to extracted text and vector-store chunks.
CHAT_HISTORY	User ID, document reference, question, generated answer and timestamp.	Stores document-chat conversations for the logged-in student.
QUIZ / QUESTION	Quiz title, source document or topic, question text, options and answer key.	A quiz contains multiple questions.
QUIZ_RESULT	User ID, quiz ID, selected answers, score and attempt time.	Connects student performance to the quiz and topic.
FLASHCARD	Document or topic reference, prompt and answer.	Stores generated revision cards.
PERFORMANCE	Topic, attempts, correct answers, accuracy and last activity.	Provides the data used for weak-topic identification.
STUDY_PLAN	User ID, exam date, available study time and planned activities.	Stores the generated revision schedule and its progress.

The first review already identifies these entities. The table above clarifies their purpose but is not a completed database schema. Primary keys, foreign keys, constraints, indexing and the final ER diagram must be prepared before backend implementation. Document chunks and embeddings will be stored in the selected vector-storage layer and linked to the corresponding DOCUMENT record through metadata.

3.6 Security and reliability design

- Use authenticated accounts and Spring Security with JWT-based API protection, as proposed in Review 1.
- Check document ownership before returning files, chat history, quiz results or study plans.
- Validate uploaded extensions, MIME types and file sizes; reject unsupported or empty files with a useful message.
- Store passwords as secure hashes and keep LLM or embedding API keys outside source code using environment configuration.
- Limit prompt context to retrieved chunks from the selected student document and retain source metadata for traceability.
- Handle LLM timeouts, rate limits and malformed responses without losing previously saved student data.

4. Technology Selection

4.1 Proposed technology stack

Component	Selection from Review 1	Reason for retaining it / implementation note
Frontend	HTML, CSS and JavaScript.	Enough for the planned student-facing pages without requiring a separate frontend framework.
Backend	Java and Spring Boot; REST APIs.	Provides one maintainable backend for authentication, document management, AI requests and learning modules.
Relational database	MySQL with JPA / Hibernate.	Suitable for structured records such as users, documents, chat history, quiz attempts and study plans.
PDF processing	Apache PDFBox.	Java library intended for reading and extracting content from PDF documents [7].
Word / PowerPoint processing	Apache POI.	Provides Java APIs for Microsoft Office document formats, including Word and PowerPoint [8].
RAG and embeddings	Embedding model/service, chunking and vector search.	Enables retrieval of relevant passages from uploaded material before LLM generation [1], [2].
Vector storage	PostgreSQL with pgvector or compatible solution; final decision pending.	pgvector supports vector similarity search in PostgreSQL [9]. The final option should be selected based on setup effort and integration with the team's environment.

Component	Selection from Review 1	Reason for retaining it / implementation note
AI generation	One LLM API provider for the first working version.	Keeps API integration and debugging manageable. Provider selection, usage limits and cost must be checked before implementation.
Security	Spring Security and JWT.	Supports authenticated API access and student-specific data protection.
Version control / deployment	Git, GitHub and Docker as proposed.	Git/GitHub will support team collaboration. Docker may be used for repeatable local setup if time permits.

4.2 Alternatives and scope decisions

Alternative / approach	Decision for the initial version	Reason
Microservices architecture	Not selected. Use a modular Spring Boot application.	Separate deployed services would add configuration, networking and deployment work without being necessary for the project scope.
Training or fine-tuning a custom LLM	Not planned. Use an existing LLM API.	Fine-tuning requires suitable datasets, compute and evaluation effort. RAG is sufficient for grounding responses in uploaded material.
Complex ML model for weak-topic prediction	Not planned initially. Use quiz accuracy and simple thresholds.	Rule-based analysis is understandable, testable and adequate for an initial student project.
Multiple LLM providers and fallback orchestration	Not planned for the first version.	A single configured provider limits integration work and makes failures easier to diagnose.
OCR for scanned handwritten documents	Excluded from the initial implementation.	OCR and handwriting recognition could substantially expand the scope. The first version will prioritize text-based files and give a clear message when extraction returns no usable text.
Advanced recommendation algorithm	Deferred. Begin with a rule-based planner.	The first-review inputs (exam date, available study hours, weak topics and progress) can support a useful basic schedule without a complex model.

These decisions preserve the project direction from Review 1 while avoiding additional components that are not essential to demonstrate the central learning cycle. A final confirmation of the vector-store option and the LLM/embedding provider is still required.

5. Implementation Progress

The implementation status below is based on the team's development that has started and on the evidence available so far. It separates requirements/design preparation from software delivery. No executable application, live AI integration or completed module is claimed in this report.

Component	Current status	Work remaining
Project scope and objectives	Documented in Review 1.	Confirm the minimum viable feature set and acceptance criteria with the guide.
Literature and research gap	Initial themes listed; a preliminary reference set is identified for this report.	Complete the literature matrix, compare existing systems and refine the research gap.
System architecture	High-level structure proposed in Review 1 and elaborated in §3.	Confirm component boundaries, request contracts and the final architecture diagram.
Database design	Initial entity list proposed.	Prepare ER diagram, fields, keys, relationships and database migrations.
Frontend	In progress; Dashboard and subsequent pages.	Create basic pages for registration/login, document library, chat, quizzes and dashboard.
Spring Boot backend	Not started.	Set up project, REST controllers, service layer, persistence and validation.
Authentication and access control	Not started.	Implement registration, login, password hashing, JWT and document ownership checks.
Document upload and extraction	Not started.	Implement upload validation and extraction for the selected supported formats.

Component	Current status	Work remaining
RAG pipeline and AI chat	Not started.	Implement chunking, embeddings, vector retrieval, prompt construction and source display.
AI learning features	Not started.	Add summary, MCQ, flashcard and viva generation after the basic RAG flow works.
Quiz, analytics and study planner	Not started.	Store attempts, calculate scores, identify weak topics and generate a rule-based schedule.
Testing and deployment	Not started.	Run module, integration and usability tests; prepare a reproducible local setup and demonstration.

5.1 Recommended minimum viable implementation

To avoid attempting all features at once, the development sequence should produce a small end-to-end version before adding secondary modules. The first demonstrable version should include:

- Basic student registration and login with authenticated access.
- Upload and list a text-based PDF, with file type and size validation.
- Extract text, divide it into chunks, generate embeddings and store searchable chunk metadata.
- Ask a question about the uploaded PDF and display a response based on retrieved passages, with source information where available.
- Generate a short summary and a small MCQ set from the selected document; store quiz attempts and show the score.

Once this vertical workflow works reliably, the team can extend document support to Word and PowerPoint, add flashcards and viva questions, and then implement previous-paper analysis, topic-wise performance and the study planner. This sequence reduces the risk of spending time on independent screens that are not yet connected to a working learning workflow.

5.2 Implementation sequence and module dependencies

Order	Implementation unit	Dependency / exit condition
1	Spring Boot project, MySQL connection, basic frontend and authentication.	A student can register, log in and access protected endpoints.
2	Document upload, file metadata and text extraction.	A supported document is stored and readable text is available to the backend.
3	Chunking, embeddings and vector indexing.	Chunks can be retrieved using a sample query and mapped back to a source document.
4	RAG-based document chat.	A question returns context-relevant output and displays available source references.
5	Summary and question generation.	The selected content produces a usable summary and structured MCQs.
6	Quiz scoring, weak-topic summary and basic study plan.	Results are stored and used to prioritize revision topics.
7	Previous paper analysis, further file formats, polish and deployment.	The core workflow remains stable while secondary features are integrated.

5.3 Anticipated defects and planned handling

No defects regarding frontend have been recorded from live execution as of yet. The following are common issues anticipated for a document-based RAG learning application. They are included as a development checklist and must not be reported as defects already encountered.

Potential issue (not yet observed)	Likely cause	Planned handling
Uploaded document produces no extracted text.	The file may be scanned, password-protected, corrupted or contain unsupported formatting.	Validate file readability; detect empty extraction output; show a clear message and request a text-based file for the first version.
Upload fails for a valid-looking file.	Extension and MIME type may not match, or the file may exceed the configured size limit.	Validate file type and size on the server; return a specific validation message and avoid creating an incomplete document record.

Potential issue (not yet observed)	Likely cause	Planned handling
RAG response is unrelated to the question.	Poor chunk boundaries, weak retrieval results or an overly broad query.	Review chunk size and overlap; inspect retrieved passages; pass only relevant context and indicate when no useful context is found.
Answer is generated without a reliable source.	The LLM may answer from its general training instead of the supplied context.	Use a context-restricted prompt, require source metadata for retrieved passages and instruct the model to say when the notes do not contain the answer.
Generated MCQ has missing options or an invalid answer key.	The LLM may return inconsistent or malformed structured output.	Request a fixed output format; validate the question, options and answer key before saving; reject invalid items with a clear error.
AI request times out or is rate-limited.	Network delay, provider quota or temporary API unavailability.	Set request timeouts, show a retryable error and avoid losing the uploaded document or prior quiz data.
A student can access another user's document or result.	An endpoint may check authentication but fail to check resource ownership.	Apply owner checks in backend service methods and add negative authorization tests before demonstration.

6. Innovation and Project Contribution

The project's contribution is primarily in the integration and student-specific use of established techniques, rather than in proposing a new language model or a new retrieval algorithm. The proposed platform connects activities that students often perform separately: understanding their own notes, preparing revision material, attempting assessments, identifying weak topics and planning the remaining study time.

6.1 Source-grounded assistance

The RAG workflow is intended to retrieve relevant passages from the student's uploaded material before asking an LLM to generate an answer. This gives the assistant a defined academic context and creates an opportunity to show which source material informed the response. Foundational RAG research describes combining a generative model with an external retrieval component [1]. Recent education-focused surveys discuss its use for educational question answering and the continuing need to address hallucination, retrieval quality and evaluation [2], [3].

6.2 Connected study cycle

The main functional contribution is the proposed sequence Upload → Understand → Practise → Analyse → Improve. A document added for question answering can also be used to generate a summary, create MCQs, populate a quiz, contribute to topic-wise performance and inform revision planning. This reduces the need for students to move between unrelated tools for each stage of exam preparation.

6.3 Practical personalization

The initial personalization mechanism is intentionally explainable. It uses the student's quiz results, identified weak topics, examination date and available study hours to order revision activities. This is a practical first step toward adaptive learning and avoids claiming that a sophisticated learner model or predictive algorithm is already available. Research on intelligent tutoring systems also emphasizes that educational value should be evaluated rather than assumed [5].

7. Technical Complexity and Feasibility

The technical difficulty is concentrated in the interfaces between the document-processing, retrieval, LLM and student-data modules. Each component is familiar in isolation, but the platform must preserve document ownership, source metadata and topic relationships as information moves through the workflow.

Source of complexity	Why it matters	Manageable approach
Document format variability	PDF, Word and PowerPoint files do not expose their text in the same way; some files may be scanned or malformed.	Begin with text-based PDFs; add Word and PowerPoint extraction after the first pipeline is stable. Use explicit validation and processing status.
Chunking and retrieval quality	Chunks that are too long or too short may retrieve irrelevant or incomplete context.	Start with a simple chunking strategy and fixed overlap; test it on a small set of course notes and adjust using observed examples.
Embedding and vector-store setup	Embedding dimensions, metadata and vector queries must remain consistent.	Choose one embedding model and one vector store; keep document/chunk identifiers with every vector record.

Source of complexity	Why it matters	Manageable approach
LLM reliability and output format	Generated answers can be unsupported, and generated questions may contain malformed options.	Use context-specific prompts, validate structured outputs and provide a safe error when output cannot be parsed.
Performance and topic association	A quiz score is useful for planning only if the attempt is linked to a topic or source section.	Attach each generated question to its topic/document and calculate initial accuracy using stored attempts.
Security across modules	Documents and learning records are private student data.	Use JWT authentication and enforce student ownership in every relevant backend service.
Scope and integration effort	The proposed scope includes many features and several external services.	Complete one end-to-end MVP first; defer OCR, custom model training, advanced recommendation algorithms and multi-provider orchestration.

8. Milestone Completion

8.1 Status against the proposed work sequence

WP	Work package	Indicative window	Status at Review 2
WP1	Problem definition, objectives and initial project proposal.	Jul–Aug 2026	Complete as documentation — captured in Review 1.
WP2	Literature survey and research-gap refinement.	Aug–Sep 2026	Partially complete — themes identified; critical literature matrix and final gap statement remain.
WP3	System architecture and database design.	Sep–Oct 2026	Partially complete — high-level architecture and entity list proposed; ERD, API contracts and final vector-store decision remain.
WP4	Backend foundation, frontend skeleton and authentication.	Sep–Oct 2026	Partially Complete — high level frontend skeleton is ready.
WP5	Document upload, extraction, chunking and RAG retrieval.	Oct 2026	Not complete yet — planned as the first major functional development phase.
WP6	AI chat, summaries, MCQs, flashcards and viva questions.	Oct–Nov 2026	Not complete yet — depends on the document-processing and RAG pipeline.
WP7	Quiz scoring, weak-topic analysis, question-paper analysis and study planner.	Nov 2026	Not complete yet — depends on generated questions and stored quiz results.
WP8	Integration testing, usability validation, documentation and final demonstration.	Nov 2026	Not started — requires a working end-to-end application.

The main schedule concern is that the project is behind the sequence implied by the initial plan: document-processing implementation should already have begun for the later AI and analytics modules to be completed comfortably. The team should therefore prioritize the minimum viable workflow, confirm the revised schedule with the guide and avoid expanding the scope before the RAG chat and quiz path is functional.

8.3 Proposed team ownership

The first-review report identifies the five team members but does not record module-wise ownership. The following allocation is a suggested starting point for discussion and should be confirmed by the team before work is divided.

Team member	Suggested primary responsibility	Supporting responsibility
Vishwajeet Yadav	Backend structure, REST API integration and overall coordination.	Integration and final system walkthrough.
Aayush Aherkar	Document upload, extraction and RAG retrieval pipeline.	Source metadata and retrieval testing.
Kshitij Rathod	Frontend pages and student interaction flow.	Usability checks and UI integration.
Brijesh Ingavale	LLM integration, document chat and learning-content generation.	Prompt/output validation.
Shiv Bhosale	Quiz records, performance analysis and study-planner logic.	Test cases, documentation and report preparation.

9. Testing and Validation Plan

Formal testing cannot be reported as completed at this stage because there is no implemented application to execute. The following test plan identifies the minimum checks required as each module is developed. Lower-level validation should be performed before connecting the LLM, so that document extraction, database and scoring errors can be isolated from model behaviour.

Test area	Test / expected result	Requires external AI service?
File validation	Accept a supported file within the configured limit; reject unsupported, oversized or empty files.	No
Text extraction	Extract readable text from a sample PDF and verify that empty extraction is detected.	No
Chunk and metadata check	Verify chunk IDs, document ownership and available page/section metadata.	No
Vector retrieval	A sample query retrieves relevant chunks from the selected document.	Embedding service may be required
RAG answer quality	The answer is consistent with retrieved passages and reports insufficient context when appropriate.	Yes
Summary and MCQ generation	Generated summary is readable; each MCQ contains valid options and an answer key.	Yes
Quiz scoring	Selected answers are evaluated correctly and the result is stored against the right quiz/user.	No
Weak-topic identification	A low-scoring topic is flagged according to the configured rule.	No
Study-plan generation	The plan respects the exam date and available study hours entered by the student.	No, for initial rule-based plan
Authorization	A student cannot view or modify another student's document, chat or result.	No

10. Updated Risk Register

Risk	Likelihood	Impact	Mitigation / current position
Implementation starts later than planned and reduces integration time.	High	High	Prioritize the MVP sequence in §5.1; assign owners and weekly exit criteria; defer non-essential features.
LLM or embedding API has usage limits or unexpected cost.	Medium	High	Choose a provider after checking quotas and pricing; limit request size; cache document processing where feasible; keep API calls configurable.
Scanned or poorly formatted documents produce little usable text.	High	Medium	Start with text-based documents; validate extraction output; communicate unsupported scanned-file cases clearly.
Retrieved passages do not answer the question accurately.	Medium	High	Test with known question-document pairs; adjust chunking and retrieval settings; show source passages and acknowledge insufficient context.
LLM returns unsupported answers or malformed MCQs.	Medium	High	Use context-restricted prompts and output validation; do not save malformed questions; require source support for document chat.
Vector-store and MySQL integration adds setup complexity.	Medium	Medium	Choose one vector option early; store stable document/chunk IDs; keep vector persistence behind a small service interface.
Sensitive student files are exposed through an API route.	Low–Medium	High	Apply JWT authentication, ownership checks and upload validation; test access denial between two sample accounts.
Advanced modules consume time before core workflow is stable.	High	High	Implement upload → retrieve → answer → quiz first; leave OCR, complex analytics and multi-provider orchestration out of scope.

11. Review 2 Presentation and Demonstration Plan

Since the software has not yet been implemented, the Review 2 presentation should be a design walkthrough rather than a live working-product demonstration. Any mock screen or sample answer shown during the presentation should be labelled as a proposed interface or illustrative example, not as output produced by the project system.

#	Presentation step	What it should establish

#	Presentation step	What it should establish
1	Explain the student problem and the integrated learning cycle.	The project scope remains aligned with Review 1: upload, understand, practise, analyse and improve.
2	Walk through the architecture diagram and module responsibilities.	The frontend, Spring Boot backend, document processor, RAG/LLM service and data stores have defined roles.
3	Trace one sample document question from upload through extraction, chunking, retrieval and answer generation.	The team understands the intended RAG data flow and the role of source metadata.
4	Show the conceptual database entities and key relationships.	The main student records and learning records have been considered before implementation.
5	Present the minimum viable implementation sequence, risk register and revised milestones.	The scope is practical, dependencies are understood and the team has an actionable recovery plan.

12. Plan Through Review 3

Window	Work	Expected evidence for Review 3
Early November 2026	Complete backend foundation, database integration, user authentication and document management. Begin document upload and text extraction.	Working application foundation with user management and basic document upload functionality.
First Half of November 2026	Implement the RAG pipeline, including document chunking, embeddings, vector storage and retrieval. Integrate the LLM API for document-based question answering.	Functional document processing pipeline and AI-powered study chat.
Mid-November 2026	Implement AI learning features, including summaries, MCQ generation, flashcards and viva questions. Begin quiz evaluation and score tracking.	Core AI learning and assessment modules integrated into the application.
Second Half of November 2026	Complete performance analytics, weak-topic identification, previous question-paper analysis and personalized study-plan generation.	Integration of the major learning, assessment and personalization modules.
Late November 2026	Conduct functional, integration and usability testing. Resolve identified defects, improve the interface and verify the complete application workflow.	Stable working application with tested core functionalities.

References

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” Advances in Neural Information Processing Systems, vol. 33, 2020. https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[2] Z. Li, Z. Wang, W. Wang, K. Hung, H. Xie and F. L. Wang, “Retrieval-augmented generation for educational application: A systematic survey,” Computers and Education: Artificial Intelligence, vol. 8, article 100417, 2025. https://doi.org/10.1016/j.caeai.2025.100417

[3] J. Swacha and M. Gracel, “Retrieval-Augmented Generation (RAG) Chatbots for Education: A Survey of Applications,” Applied Sciences, vol. 15, no. 8, article 4234, 2025. https://doi.org/10.3390/app15084234

[4] N. Mulla and P. Gharpure, “Automatic question generation: a review of methodologies, datasets, evaluation metrics, and applications,” Progress in Artificial Intelligence, vol. 12, pp. 1–32, 2023. https://doi.org/10.1007/s13748-023-00295-9

[5] A. Létourneau et al., “A systematic review of AI-driven intelligent tutoring systems (ITS) in K-12 education,” npj Science of Learning, vol. 10, article 29, 2025. https://doi.org/10.1038/s41539-025-00320-7

[6] A. K. Bhowmick et al., “Automating question generation from educational text,” arXiv:2309.15004, 2023. https://arxiv.org/abs/2309.15004

[7] Apache Software Foundation, “Apache PDFBox — A Java PDF Library,” project documentation. https://pdfbox.apache.org/

[8] Apache Software Foundation, “Apache POI — Component Overview,” project documentation. https://poi.apache.org/components/index.html

[9] pgvector contributors, “pgvector: open-source vector similarity search for Postgres,” project documentation. https://github.com/pgvector/pgvector

[10] VMware / Broadcom, “Spring Boot Reference Documentation,” project documentation. https://docs.spring.io/spring-boot/index.html

